

torch.bmm#

<https://pytorch.org/docs/stable/generated/torch.bmm.html>

Description:

Performs a batch matrix-matrix product of matrices stored in input and mat2. input and mat2 must be 3-D tensors each containing the same number of matrices. If input is a $(b \times n \times m)$ ($b \times n \times m$) tensor, mat2 is a $(b \times m \times p)$ ($b \times m \times p$) tensor, out will be a $(b \times n \times p)$ ($b \times n \times p$) tensor. This operator supports TensorFloat32. On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

Function Signature

```
torch.bmm(input, mat2, out_dtype=None, *, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> input = torch.randn(10, 3, 4)
>>> mat2 = torch.randn(10, 4, 5)
>>> res = torch.bmm(input, mat2)
>>> res.size()
torch.Size([10, 3, 5])
```

Notes & Warnings

NOTE

This function does not broadcast. For broadcasting matrix products, see `torch.matmul()`.

Detailed Documentation

Documentation

Performs a batch matrix-matrix product of matrices stored in input and mat2.

input and mat2 must be 3-D tensors each containing the same number of matrices.

If input is a $(b \times n \times m)$ ($b \times n \times m$) tensor, mat2 is a $(b \times m \times p)$ ($b \times m \times p$) tensor, out will be a $(b \times n \times p)$ ($b \times n \times p$) tensor.

This operator supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

This function does not broadcast. For broadcasting matrix products, see torch.matmul().

input (Tensor) – the first batch of matrices to be multiplied

mat2 (Tensor) – the second batch of matrices to be multiplied

out_dtype (dtype, optional) – the dtype of the output tensor, Supported only on CUDA and for torch.float32 given torch.float16/torch.bfloat16 input dtypes

out (Tensor, optional) – the output tensor.